Aprendizaje automático II

Guía de estudio

Manuel Castillo-Cara, Luis M. Sarro
{manuelcastillo,lsb}@dia.uned.es

10-02-2026

En esta asignatura tomaremos como libro de referencia *The Elements of Statistical Learning: Data Mining, Inference, and Prediction*, de T. Hastie, R. Tibshirani y J. Friedman (a partir de ahora nos referiremos a él como [EST]). Este libro puede descargarse gratuitamente de la web del primer autor en la universidad de Stanford (`https://web.stanford.edu/~hastie/ElemStatLearn/`), y, puesto que cubre mucho más contenido del que se corresponde con el contenido de la asignatura, en cada tema indicaremos los epígrafes que es necesario estudiar.

La elección de este texto de referencia responde a varias consideraciones que esperamos que acabéis compartiendo con nosotros. Pese a que sabemos perfectamente que puede resultar un libro intimidante al principio, tiene también algunas ventajas incuestionables. Por un lado, su licencia libre permite que lo consigáis sin pagar (aunque a muchas de las personas que lo estudiamos nos acaba pareciendo buena idea tenerlo en papel). Por otro lado, su enfoque matemático consigue un objetivo esencial, en nuestra opinión, para la Ciencia de Datos: que no nos olvidemos de que los métodos y algoritmos que vemos en la asignatura no son construcciones mágicas sino dispositivos matemáticos enraizados en la teoría.

Además, para facilitar la comprensión de algunos conceptos que no están explicados suficientemente en dicho libro, utilizaremos también abundante bibliografía complementaria, que indicaremos en este documento al principio de cada tema. Hemos aplicado el criterio de que cada elemento de esta bibliografía complementaria debía estar, como el propio libro [EST], disponible para su consulta gratuita en internet.

Finalmente, resaltar que se ha puesto a disposición del alumnado las prácticas de programación relativas a las cuatro áreas temáticas del curso. Estas prácticas de programación están diseñadas en Python y se encuentran en mismo aula virtual. Cabe resaltar que se recomienda encarecidamente desarrollar los conocimientos teóricos del libro [EST] en las prácticas de programación de Aprendizaje Automático II.

1. Bosques aleatorios

En este tema abordaremos el estudio del método estadístico de *bootstrap* y su aplicación para reducir la varianza de algoritmos de aprendizaje automático, en la técnica de agregación de *bootstrap* llamada *bagging*. De ahí, introduciremos los bosques aleatorios, un modelo enormemente exitoso y sencillo de utilizar, pero con particularidades matemáticas que lo hacen muy interesante.

1.1. Bibliografía complementaria

- [ISL] *An introduction to statistical learning*, G. James, D. Witten, T. Hastie and R. Tibshirani: `https://www.statlearning.com/`
- [URF] *Understanding Random Forests: from Theory to Practice*, Gilles Loupe, Univ. de Liège: `https://arxiv.org/abs/1407.7502`
- [WIB] What Is Bootstrapping in Statistics? `https://www.thoughtco.com/what-is-bootstrapping-in-statistics-3126172`
- [RF] *Random Forests*. Breiman, L. Machine Learning (2001) 45: 5. `https://doi.org/10.1023/A:1010933404324`

1.2. Contenido

Este bloque inaugura el estudio de la **taxonomía ensemble** o de agregación de modelos, que vertebra los tres primeros temas de la asignatura: *Bagging*, *Boosting* y *Stacking*. En este bloque nos centraremos en las técnicas basadas en *bootstrap aggregation*, o *bagging*, en especial Bosques Aleatorios.

Descomposición en sesgo/varianza

- Antes de abordar el estudio de cualquier modelo, es importante entender que su error puede descomponerse en diferentes componentes. Este epígrafe se dedica a su análisis.
- Para una comprensión inicial, resulta útil repasar la discusión sobre sesgo y varianza en *[ISL 2.2.2]*, ya estudiada en la asignatura *Modelado Estadístico de Datos*. En esta asignatura, profundizaremos en estos conceptos a través de *[ESL 7.1–7.3]*.
- Es esencial comprender la diferencia entre el **error de test (o de generalización), el error de predicción esperado y el error de entrenamiento**, tanto en problemas de regresión como de clasificación (ver hilo del Foro *"Diferencia entre train/validation/test"* en Tema 1). También es importante distinguir entre los dos objetivos típicos del cálculo de error: **selección de modelos y evaluación de modelos**.
- Se estudiará la descomposición del error en sus componentes básicos, centrándonos en la Ecuación 7.9, que se puede derivar para distintos tipos de modelos.
- Como ejemplo, el libro deriva ese resultado para el método de los k-Nearest Neighbors (k-NN), para la regresión lineal y algunas de sus variantes. La Figura 7.2 esquematiza los conceptos: entenderla bien es garantía de haber comprendido lo que se plantea en este epígrafe.
- Así mismo, también es útil repasar con detenimiento el Ejemplo 7.3.1, que muestra cómo generalmente es necesario asumir un cierto equilibrio entre el sesgo y la varianza.
- Una vez estudiada esta parte en *[ESL]*, es interesante recurrir a *[URF 4.1]*, donde se explican los mismos conceptos de una manera ligeramente distinta. En concreto, es útil estudiar las Figuras 4.1 y 4.2, que han de ser entendidas perfectamente si se han comprendido los conceptos anteriores.

El método de *bootstrap*

- Es útil tener nociones de en qué consiste el método estadístico de *Bootstrap*. Para una descripción intuitiva aunque superficial, es suficiente leer *[WIB]*, donde se explica también el origen de su -intraducible- nombre. Una descripción más profunda puede seguirse en *[ISL 5.2]*.

Agregación de *bootstrap*: *Bagging*

- El método de *Bootstrap* fue diseñado para determinar la precisión de la estimación de un parámetro o de una predicción concreta.
- Sin embargo, también es posible utilizarlo para mejorar las propias estimaciones y predicciones. En esto consiste el *Bagging*. La idea clave es que si promediamos un conjunto de observaciones, podemos reducir la varianza.
- Pese a que el término *Bagging* proviene de la contracción de *Bootstrap Aggregation*, su colisión con el sustantivo *bag* y sus derivados ha tenido éxito porque, de alguna manera, es posible concebir el *bagging* como una técnica basada en "embolsar" de distintas maneras un mismo conjunto de datos. Así es frecuente encontrar el verbo *to bag* para referirse a la aplicación del *Bagging*.
- Una descripción somera de esta técnica es la ofrecida en *[ISL 8.2.1]*, y se recomienda su estudio en primer lugar.
- A continuación es útil volver a *[ESL 8.7]*, donde, tras otra descripción breve de la técnica, se ilustra primero con un ejemplo sencillo centrado en un modelo de *splines* que es menos interesante que el siguiente ejemplo, basado ya en árboles y que es conveniente comprender en sus detalles.
- A continuación es clave estudiar el más detallado ejemplo de *[ESL 8.7.1]*, donde se estudian las propiedades matemáticas del *Bagging* para un sencillo problema de clasificación, que muestra una idea básica que subyace a este método: promediar reduce la varianza y deja el sesgo inalterado en el caso de que se use una función de pérdida basada en el error cuadrático (Ecuación 8.52).

Bosques aleatorios

- Tras haber estudiado los conceptos relacionados, estamos ya en condiciones de abordar el estudio del modelo que da título a este tema. Para ello, seguiremos la discusión propuesta en *[ESL 15]*. El método de los bosques aleatorios es una inteligente modificación del método de *Bagging* que consiste en construir una colección grande de árboles no-correlados entre sí mediante la introducción de perturbaciones aleatorias en el proceso inductivo, para luego promediar sus resultados.
- La principal razón del éxito de los bosques aleatorios es que ofrecen resultados buenos en multitud de casos, y son muy sencillos de construir y entrenar.
- Dado que en *[ESL]* se explica antes las técnicas de intensificación (*boosting*) que los bosques aleatorios, será necesario ignorar las referencias a las primeras en el estudio de este tema. Una vez que se estudie el tema dedicado a la intensificación (Tema 2) será útil volver a repasar *[ESL 15]*.

- Una vez estudiada y comprendida la descripción de los bosques aleatorios *[ESL 15.2]*, es importante dedicar tiempo a comprender algunos de sus detalles más relevantes, analizados en *[ESL 15.3]*:
  - Las muestras «fuera de la bolsa» permiten aproximar el error de una forma parecida a como lo haríamos con validación cruzada, pero con la ventaja de que en este caso puede hacerse al mismo tiempo que se construye el modelo.
  - También es posible obtener una aproximación a la importancia relativa de las distintas variables que intervienen en el modelo. En este sentido, se espera que el alumnado sepa distinguir las distintas medidas de importancia (Gini y aleatorización).
  - Finalmente, la discusión acerca del sobreentrenamiento en bosques aleatorios aporta algunas claves esenciales para el uso práctico de esta técnica.
- En *[ESL 15.4]* se profundiza en los mecanismos que intervienen en la aleatorización que realiza el método de bosques aleatorios, especialmente con relación a la varianza y al sesgo. No es necesario el estudio de esta sección: se recomienda su abordaje solo en la medida en que ayude a asentar los conceptos de las anteriores secciones.
- Finalmente, como fuente complementaria, puede ser interesante acudir al artículo que la comunidad científica considera como el origen de los bosques aleatorios, *[RF]*.

1.3. Prácticas de programación

- Visualizar los contenidos digitalizados de las prácticas de programación de Aprendizaje Automático II correspondientes a los módulos *"Modelos Múltiples"* y *"Bagging"*

2. Intensificación (*boosting*)

La intensificación o *boosting* es una familia de algoritmos que construyen modelos generales "fuertes" a partir de modelos parciales (o "aprendices débiles"). En este tema abordaremos este paradigma de agregación de modelos que permiten mejorar las predicciones de cualquier algoritmo de aprendizaje automático.

2.1. Bibliografía

- [ISL] *An introduction to statistical learning*, G. James, D. Witten, T. Hastie and R. Tibshirani: `https://www.statlearning.com`
- [BFA] *Boosting: Foundations and Algorithms*, Robert E. Schapire, Yoav Freund. `https://direct.mit.edu/books/oa-monograph/5342/BoostingFoundations-and-Algorithms`
- [PML] *Practical machine learning with H2O*, Darren Cook: `https://www.oreilly.com/library/view/practical-machine-learning/9781491964590/`
- [CLX] *CatBoost vs. LightGBM vs. XGBoost*,: `https://towardsdatascience.com/catboost-vs-lightgbm-vs-xgboost-c80f40662924/`

2.2. Contenido

Introducción

- En este tema utilizaremos el término **Intensificación** como equivalente al término inglés ***Boosting***, tal como lo aplicamos en los bloques 2 y 3 del curso.
- La intensificación es otra aproximación, muy distinta de los bosques aleatorios, para mejorar las capacidades predictivas de cualquier modelo sencillo, y también de un árbol de decisión. Una buena manera de iniciarse en los conceptos implicados es leer *[ISL 8.2.3]*, donde, en unos pocos párrafos, se explican estos conceptos.

AdaBoost

- Podríamos decir que *[BFA]* es la «biblia» de la intensificación adaptativa. El libro cubre con detalle aspectos muy específicos de este enfoque, y es de obligada lectura para quien quiera conocerlo y dominarlo a fondo. Si bien el libro se centra en problemas de clasificación, la adaptación a problemas de regresión es inmediata una vez que se conoce el mecanismo esencial. En esta asignatura nos quedaremos en un nivel mucho más superficial.
- Hay muchos algoritmos que hacen uso de la idea de intensificación. En *[BFA 1.2]* se expone esta idea y se describe uno de los algoritmos más exitosos y conocidos: AdaBoost (contracción de *Adaptive Boosting*). Es importante seguir la discusión del ejemplo «de juguete» *[BFA 1.2.1]*, que clarifica el funcionamiento del algoritmo. También es interesante analizar el ejemplo sobre el diagnóstico médico *[BFA 1.2.3]*.

Intensificación de gradiente

- La idea básica de AdaBoost es asignar un peso a cada uno de los ejemplos del conjunto de entrenamiento, e ir progresivamente incrementando los pesos para aquellos ejemplos que resultan difíciles en cada iteración (intensificando la atención de los modelos sucesivos sobre esos ejemplos). Otra manera distinta de aplicar la idea de intensificación es la intensificación de gradiente. En este caso, en lugar asignar pesos mayores a los ejemplos mal clasificados en la iteración anterior, se identifican estos ejemplos mediante los residuos de esa iteración, y se trata de minimizarlos en la siguiente. En otras palabras, cada nueva iteración intenta modelizar (y minimizar) los errores de la anterior.
- Para entender la diferencia entre AdaBoost y la intensificación de gradiente, es necesario primero estudiar con detalle *[ESL 10.9]*, donde se formaliza la operación de intensificación en árboles y se explica cómo de la elección de la función de pérdida depende el tipo de intensificación. Una vez asentados estos conceptos, *[ESL 10.10]* explica con detalle la intensificación de gradiente.
- Es interesante también en *[ESL 10.13]* la discusión sobre la posibilidades de interpretabilidad que tienen los métodos basados en intensificación: como en los bosques aleatorios, es posible calcular la importancia de las variables, y, además, se pueden analizar las interacciones entre las variables mediante los gráficos de dependencias parciales.
- Finalmente, en *[ESL 10.14]*, se ilustran estos métodos mediante tres ejemplos muy completos.

2.3. Implementaciones de la intensificación de gradiente

- Existen multitud de implementaciones de este algoritmo, muchas de ellas de libre acceso. Cuatro de las más conocidas son AdaBoost, XGBoost, GBM y LightGBM, y en en las prácticas de programación se ofrece un ejemplo de sus fundamentos e implementación en Scikit-learn. En *[CLX]* se ofrece una comparativa entre ellas.

2.4. Prácticas de programación

- Visualizar los contenidos digitalizados de las prácticas de programación de Aprendizaje Automático II correspondientes al módulo *"Boosting"*.
- Enfoque práctico complementario. Para un enfoque centrado en el uso de la intensificación de gradiente, con ejemplos prácticos acerca de cómo elegir los parámetros del algoritmo, ver *[PML 6]*, además de las propias prácticas de programación.

3. Otras combinaciones de modelos

En este tema abordaremos las técnicas de reducción de varianza englobadas bajo el término *ensemble learning*. Para ello haremos uso de la idea de compromiso entre sesgo y varianza, y mostraremos cómo es posible reducir la varianza de modelos que suelen presentarla alta mediante la agregación de conjuntos de esos mismos modelos.

3.1. Bibliografía

- [CCT] *Classifiers Combination Techniques: A Comprehensive Review*, M. Mohandes, M. Deriche and S.O. Aliyu: `https://ieeexplore.ieee.org/document/8335271`
- [CCS] *Is Combining Classifiers with Stacking Better than Selecting the Best One?*, S. Džeroski and B. Ženko: `https://doi.org/10.1023/B:MACH.0000015881.36452.6e`

3.2. Contenido

Introducción

- En este tema abordaremos otras formas de combinar modelos distintas al *Bagging* y a la intensificación. A diferencia de estos dos métodos, en los que la idea era combinar modelos básicos o «débiles», otra manera de abordar los problemas de aprendizaje automático consiste en consultar a distintos expertos y combinar sus predicciones: esta estrategia generalmente produce mejores resultados que los obtenidos por cada uno de ellos.
- Como pasa en muchos otros ámbitos del aprendizaje automático, este área es relativamente nueva y no está muy sistematizada (de hecho, ni siquiera hay consenso en el nombre que engloba estas técnicas). Es el motivo, por ejemplo, de que en *[ESL]* las técnicas de combinación estén dispersas por varios epígrafes: además de los dedicados al *Bagging* y la intensificación (vistos en el tema 1 y 2) en el epígrafe 8.8 se habla de la técnica de generalización apilada (*stacked generalization*) y en el capítulo 16 se exponen los detalles de otro enfoque basado en árboles: la intensificación regularizada y sus variantes.

- A fin de proporcionar un mapa más general y actualizado de las posibilidades de combinación de modelos que han sido propuestas, en este tema nos basaremos en lo expuesto en *[CCT]*, y será exclusivamente este el contenido que evaluaremos. Este artículo, de 2018, contiene la revisión mas moderna y exhaustiva de métodos de combinación que puede encontrarse. Presenta una taxonomía de técnicas en función de sus características comunes, y para cada clase discute las ventajas e inconvenientes además de su complejidad computacional y áreas de aplicación.
- Conviene aclarar que *[CCT]* se centra exclusivamente en modelos diseñados para resolver problemas de clasificación. Las ideas que son de aplicación en esos problemas son casi siempre directamente traducibles a problemas de regresión.
- La introducción de *[CCT]* arranca explicando de forma breve la idea de combinación de modelos y pasa enseguida a hacer una revisión de aquellos trabajos dedicados a, justamente, revisar el «estado del arte» (según Fundéu, es preferible usar «estado de la técnica») en este ámbito. No es necesario estudiar esta revisión, pero es interesante leerla en todo caso.

Marco general para la combinación de clasificadores

- La sección II de *[CCT]* dibuja el marco general para la combinación de clasificadores. Hace uso de la idea de «nota» o «calificación» (*score*) de un clasificador, que se refiere a un número que codifica cómo de bien o mal se comporta dicho clasificador con cada clase. Entonces, la combinación de clasificadores consiste en encontrar una función que acepte vectores N-dimensionales de calificación para cada uno de los M clasificadores, produciendo a partir de ellos una clasificación única. Esto queda ilustrado en la Figura 1. A continuación se ofrece la notación matemática, en el marco bayesiano, para lo mismo. El último párrafo de esta sección es especialmente interesante.

Estrategias para la combinación de clasificadores

- La sección III contiene el núcleo del artículo, mostrando una revisión detallada de las técnicas de combinación que pueden encontrarse en la literatura especializada. Para trazar la taxonomía de estas técnicas, estas son agrupadas en función de sus características comunes.
- Dichas características comunes pueden establecerse, por ejemplo, en función del nivel en el que se realice la combinación: un primer nivel temprano de «sensores», un nivel de variables/características y un tercer nivel de combinación tardío relacionado con las decisiones. La Figura 2 clarifica estas tres categorías principales y los tipos de técnicas que pueden encuadrarse en cada una.
- La categoría más frecuente y más exitosa es la combinación en el nivel de decisión, es decir, fusionar información una vez que los clasificadores han producido ya su etiquetado. La idea es justamente agregar un conjunto de clasificadores, algunos de los cuales pueden no ser muy buenos para ejemplos concretos. En esta categoría hay 3 tipos de técnicas:
    - basadas en «calificaciones» (*scores*): cada clasificador, además de la clase a la que pertenece un ejemplo de entrada, produce una medida de la incertidumbre de su decisión, que es utilizada a la hora de combinar la salida de varios clasificadores;
    - basadas en posiciones o ránkings: cada clasificador ordena de más plausible a menos las etiquetas que pueden ser asignadas a un ejemplo concreto, y se combinan estos ránkings para obtener uno general;

    - basadas en combinación abstracta: las etiquetas únicas propuestas por cada clasificador son usadas directamente como entradas del sistema de combinación.

- A continuación, los siguientes párrafos tratan un fundamento teórico de las técnicas de combinación basadas en el nivel de decisión, la teoría de evidencias o D-S. No es necesario estudiar esta teoría.

- La siguiente subsección muestra otra forma de categorizar las técnicas de combinación de clasificadores, en función de si dicha combinación se hace mediante un «umbral duro» o uno «blando». El primer caso se refiere a que a la hora de decidir la salida global de un conjunto de clasificadores, se plantea como una votación: aquella clase votada por más clasificadores será la clase elegida como salida global (ver Figura 4). Esta votación puede ser de varios tipos y el sistema admite otras mejoras. El segundo caso, por el contrario, es más complicado, ya que utiliza aproximaciones a la probabilidad a posteriori de la clase. Dependiendo de cómo se agregue esta probabilidad, existen también diversas variantes.

- Hay una tercera manera de categorizar las técnicas de combinación de clasificadores: según sean adaptativas o no. En las no adaptativas, la decisión se toma siempre de la misma forma: sea por mayoría simple, por mayoría cualificada, mediante el conteo Borda etcétera. Las adaptativas, también conocidas como técnicas de apilamiento o *Stacking*, en cambio, modifican el patrón de agregación en función de las características de cada ejemplo. Entran en juego aquí los algoritmos de inteligencia artificial, que pueden ser entrenados para modificar la agregación de forma dinámica y adaptada a los datos.

- Es interesante leer la revisión de técnicas de esta última categoría, pues son las más recientes y las más elaboradas. Una lectura detallada de la Sección III.C es imprescindible, y además es recomendable ir un paso más allá y elegir alguno de los artículos que se reseñan para leerlo también a su vez. También, si hay interés en seguir indagando en esta línea, es recomendable leer *[CCS]*.

- Finalmente, los autores proponen una cuarta forma de categorizar los métodos de combinación según si el número de clasificadores que son combinados es bajo (<10) o alto. En el segundo caso, estaríamos en el territorio de los modelos vistos en los temas 1 y 2 de esta asignatura.

- Pese al esfuerzo de abstracción que conlleva la taxonomía propuesta en este artículo, aún quedan algunos métodos que no caen en niguna de las categorías propuestas. Estos son abordados en la Sección III.D.

- Como la mayor parte de las revisiones del estado de una técnica concreta, el artículo termina con una discusión centrada en las líneas de investigación más prometedoras. Para completar lo aprendido en este capítulo, resulta interesante conocer las perspectivas que parecían más interesantes en 2018.

3.3. Prácticas de programación

- Visualizar los contenidos digitalizados de las prácticas de programación de Aprendizaje Automático II correspondientes al módulo *"Stacking"*.

4. Aprendizaje no supervisado

En este tema abordaremos el estudio de aquellas técnicas de aprendizaje automático que permiten abordar la búsqueda de patrones en conjuntos de datos no etiquetados. Veremos la taxonomía de estas técnicas y las diferentes métricas de evaluación interna y externa de los resultados.

4.1. Bibliografía

- [EMUL] Informe técnico *Evaluation Metrics for Unsupervised Learning Algorithms*: `https://doi.org/10.48550/arXiv.1905.05667`

4.2. Contenido

El tema de Clasificación no supervisada se sigue mediante la lectura de *[ESL 14.1]*, *[ESL 14.3]* y de *[EMUL]*.

La clasificación no supervisada representa un problema radicalmente distinto del que se ha abordado tanto en la asignatura de Aprendizaje Automático I como en el resto de temas de esta asignatura. Mientras que los problemas de clasificación supervisada o regresión asumen una relación entre el espacio de entrada y el de salida (variables discretas en clasificación o continuas en regresión) en los problemas de clasificación no supervisada se pretende inferir ese espacio de salida y la relación existente entre entradas y salida. Aquí nos restringimos (como ya indica el nombre del área) a problemas de clasificación y lo que pretendemos es establecer una taxonomía de los datos: qué tipos de datos (clases) existen y cómo se definen. El objetivo, expresado en términos informales, es agrupar datos similares entre sí en una clase y separar conjuntos de datos diferentes en clases diferentes. Los términos centrales en esta definición informal son *similar* y *diferente*. De cómo definamos matemáticamente esos términos y de cómo realicemos el agrupamiento a partir de esas medidas de similitud/diferencia dependerán los resultados obtenidos. La Sección 14.1 realiza una presentación formal (y probabilista) del problema de clasificación no supervisada.

La Sección 14.3 comienza discutiendo precisamente medidas de similitud/diferencia entre patrones (Sección 14.3.1). Esas medidas dependen del tipo de variables que estemos manejando (continuas, categóricas y ordinales). A continuación discute como combinar las medidas de similitud en las diferentes dimensiones (atributos) que constituyen los patrones.

Después, la sección describe algunos métodos simples y clásicos de clasificación no supervisada. Por motivos de extensión el temario no puede contemplar otras técnicas más sofisticadas como el clustering espectral o en subespacios. Las técnicas abordadas en el temario son el agrupamiento combinatorio (rápidamente impracticable para conjuntos de datos típicos), el algoritmo denominado K-means (o K-medias), la mezcla de Gaussianas (que es la generalización probabilística del algoritmo de K-medias), la cuantización de vectores, la versión de K-medias conocida como K-medoids y el clustering jerárquico (aglomerativo o divisivo).

Como hemos indicado antes, se trata de una selección de técnicas clásicas del área. El estudiante tiene a su disposición tutoriales sobre dos técnicas avanzadas en el repositorio de documentos del curso virtual. Recomendamos especialmente el tutorial sobre técnicas espectrales de Ultike von Luxburg.

Finalmente *[EMUL]* hace repaso de la taxonomía de medidas de evaluación de agrupamientos (clasificaciones no supervisadas) en la bibliografía. Mientras que medir la bondad de un modelo de clasificación supervisada es relativamente directo (aunque lleno de sutilezas y peligros) puesto que disponemos de ejemplos cuya clase es conocida, medir la bondad de un agrupamiento, decidir el número de grupos o comparar dos agrupamientos diferentes es una cuestión difícilmente objetivable. Dependiendo de la medida de evaluación obtendremos unos resultados u otros y, en general, será difícil seleccionar un resultado. Sólo en casos en los que los patrones de entrada se encuentren claramente agrupados en grupos compactos separados entre sí por distancias grandes comparadas con la dispersión intrínseca obtendremos respuestas claras y consistentes entre las diferentes medidas de evaluación. Nuestra recomendación consiste en utilizar varias técnicas de agrupamiento y medidas de evaluación y buscar soluciones estables (soluciones consistentes entre diferentes algoritmos y bien

evaluadas por diferentes métricas) teniendo siempre en cuenta que pueden no existir.

4.3. Prácticas de programación

- Visualizar los contenidos digitalizados de las prácticas de programación de Aprendizaje Automático II correspondientes al módulo *"Aprendizaje No Supervisado"*.